

CEFSA – Centro Educacional da Fundação Salvador Arena

FESA – Faculdade Engenheiro Salvador Arena

São Bernardo do campo, 27 de maio de 2026

Enzo Brito Alves de Oliveira – RA: 082220040

Erikson Vieira Queiroz – RA: 082220021

Heitor Santos Ferreira – RA: 081230042

William Santim – RA: 082220033

Engenharia de Computação – Oitavo (8º) Semestre – EC8

Professor Dr. Vinicius Borges

Sistemas Operacionais

***Atividade N2 B2 Parte 02 – Detecção e Prevenção de Impasses
em C***

SÃO PAULO

2026

Questão 1 – Transferência bancária com ordem invertida

Ra.Q1.: Sim, há uma clara possibilidade de impasse (*deadlock*). O problema ocorre devido à intercalação da execução das *threads* e à ordem como elas pedem os recursos. A função *transferencia_1* bloqueia a *conta1* e logo em seguida entra em estado de pausa por 1 segundo (*sleep(1)*). Nesse meio tempo, a função *transferencia_2* é executada, bloqueia a *conta2* e tenta bloquear a *conta1*. Como a *conta1* já está retida pela primeira *thread*, a segunda *thread* fica esperando. Quando a primeira *thread* acorda do *sleep*, ela tenta adquirir a *conta2*, que está retida pela segunda *thread*. Ambas ficam aguardando a liberação do recurso que a outra possui, travando o sistema permanentemente.

Rb.Q1.: Todas as quatro condições (condições de *Coffman*) para a ocorrência de *deadlock* estão perfeitamente estabelecidas neste código:

- **Exclusão mútua:** As contas são recursos críticos protegidos pelas variáveis *pthread_mutex_t*, o que garante que apenas uma *thread* possa acessar uma conta específica por vez.
- **Posse e espera (*Hold and Wait*):** Uma *thread* adquire e "segura" um *mutex* (por exemplo, a primeira *thread* adquire a *conta1*) enquanto aguarda a liberação do próximo recurso necessário (a *conta2*).
- **Não-preempção:** O bloqueio não pode ser interrompido ou retirado à força pelo sistema operacional; a própria *thread* que detém o recurso precisa chamar *pthread_mutex_unlock* para liberá-lo voluntariamente.
- **Espera circular:** Forma-se um ciclo exato de dependências. A *thread* 1 possui a *conta1* e espera pela *conta2*, enquanto a *thread* 2 possui a *conta2* e espera pela *conta1*.

Rc.Q1.: A maneira mais eficaz e recomendada de prevenir esse problema é quebrar a condição de espera circular. Para fazer isso, é necessário estabelecer uma ordem global estrita de aquisição de recursos. Na prática, isso significa que todas as *threads* no programa inteiro devem sempre adquirir os *mutexes* na

mesma ordem exata, independentemente do sentido da transferência bancária. Se o sistema sempre forçar o bloqueio da *conta1* antes da *conta2*, o ciclo de espera se torna impossível de acontecer.

Rd.Q1.:

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-bank-transfers-with-reverse-order.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

// Initialize mutexes for the two bank accounts
pthread_mutex_t conta1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t conta2 = PTHREAD_MUTEX_INITIALIZER;

void *transferencia_1(void *arg)
{
    // Thread 1 acquires locks following the global lock order: conta1
    // first, then conta2.
    pthread_mutex_lock(&conta1);

    // Simulating some processing or network delay
    sleep(1);

    pthread_mutex_lock(&conta2);

    // Critical section: both resources are safely acquired
    printf(" Transferencia 1 concluida \n");

    // Unlocking resources. Doing it in the reverse order of acquisition
    // is a best practice.
    pthread_mutex_unlock(&conta2);
    pthread_mutex_unlock(&conta1);

    return NULL;
}

void *transferencia_2(void *arg)
{
    // FIX: Enforcing global lock order to prevent circular wait
    // (Deadlock).
    // Regardless of the transfer direction, we MUST acquire conta1
    // BEFORE conta2.
```

```

pthread_mutex_lock(&conta1);

// Even with a delay here, a deadlock is now mathematically
impossible
// because thread 1 would be waiting for conta1, not holding it.
sleep(1);

pthread_mutex_lock(&conta2);

// Critical section
printf(" Transferencia 2 concluida \n");

// Unlocking resources safely
pthread_mutex_unlock(&conta2);
pthread_mutex_unlock(&conta1);

return NULL;
}

int main()
{
    pthread_t t1, t2;

    // Create the two threads to simulate concurrent bank transfers
    pthread_create(&t1, NULL, transferencia_1, NULL);
    pthread_create(&t2, NULL, transferencia_2, NULL);

    // Wait for both threads to finish execution before exiting the main
program
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf(" Todas as transferencias foram concluidas com sucesso! \n");

    return 0;
}

```

Questão 2 – Três threads e ciclo completo

Ra.Q2.: Sim, constata-se a presença irrefutável de espera circular. O fenômeno materializa-se porque a *thread t1* retém o recurso *r1* e aguarda a concessão de *r2*; simultaneamente, a *thread t2* detém *r2* e postula o acesso a *r3*; por fim, a *thread t3* reserva *r3* e demanda *r1*. Esse intertravamento sequencial estabelece um ciclo topológico fechado de dependências sistêmicas, inviabilizando o progresso de qualquer uma das unidades de processamento.

Rb.Q2.: Sim, as quatro condições necessárias e suficientes de *Coffman* para a ocorrência de um impasse manifestam-se inequivocamente na estrutura atual do programa. A exclusão mútua é garantida pelo uso intrínseco das primitivas *pthread_mutex_t*. A posse e espera evidencia-se no momento em que cada thread adquire e mantém um *lock* enquanto tenta requisitar o subsequente. A não-preempção é inerente ao modelo de concorrência *POSIX* adotado, o qual impede a revogação forçada de um *mutex* detido por uma *thread*. Por fim, a espera circular consoma-se por meio da cadeia de bloqueios cíclicos delineada na assertiva anterior.

Rc.Q2.: A prevenção mais robusta para esta anomalia consiste na quebra da condição de espera circular mediante a imposição de uma ordem global estrita de aquisição de recursos. Ao estabelecer uma hierarquia determinística — determinando, por exemplo, que todos os processos solicitem os recursos inexoravelmente na sequência unificada de seus identificadores (*r1*, seguido de *r2* e, ulteriormente, *r3*) —, erradica-se a possibilidade matemática de intersecções cíclicas, mitigando o risco de impasse por elaboração arquitetural estruturada.

Rd.Q2.:

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-three-threads-and-complete-cicle.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

// Initialize mutexes representing the three shared resources
pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t r3 = PTHREAD_MUTEX_INITIALIZER;

void *t1(void *arg)
{
```

```

    // Thread 1 acquires resources following the global order: r1 then r2
    pthread_mutex_lock(&r1);
    sleep(1);
    pthread_mutex_lock(&r2);

    printf("t1 executou \n");

    // Unlocking resources in the reverse order of acquisition
    pthread_mutex_unlock(&r2);
    pthread_mutex_unlock(&r1);

    return NULL;
}

void *t2(void *arg)
{
    // Thread 2 acquires resources following the global order: r2 then r3
    pthread_mutex_lock(&r2);
    sleep(1);
    pthread_mutex_lock(&r3);

    printf("t2 executou \n");

    pthread_mutex_unlock(&r3);
    pthread_mutex_unlock(&r2);

    return NULL;
}

void *t3(void *arg)
{
    // FIX: Thread 3 must also strictly adhere to the global lock
    ordering.
    // Instead of locking r3 then r1, it MUST lock r1 before r3.
    // This breaks the circular wait condition and mathematically
    prevents deadlocks.
    pthread_mutex_lock(&r1);

    // The delay simulates concurrent execution overlap, but the system
    is now structurally safe
    sleep(1);

    pthread_mutex_lock(&r3);

    printf("t3 executou \n");

    // Best practice: unlock in the exact reverse order of acquisition
    pthread_mutex_unlock(&r3);
    pthread_mutex_unlock(&r1);
}

```

```

    return NULL;
}

int main()
{
    pthread_t thread1, thread2, thread3;

    // Instantiate and launch the three threads concurrently
    pthread_create(&thread1, NULL, t1, NULL);
    pthread_create(&thread2, NULL, t2, NULL);
    pthread_create(&thread3, NULL, t3, NULL);

    // Synchronize the main process, waiting for all threads to
    successfully complete
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    pthread_join(thread3, NULL);

    printf("Todas as execucoes foram concluidas sem impasses!\n");

    return 0;
}

```

Questão 3 – Função auxiliar que esconde risco

Ra.Q3.: A complexidade analítica decorre do encapsulamento das primitivas de sincronização em sub-rotinas distintas e aninhadas. Ao invés de as aquisições de recursos figurarem explicitamente em um único escopo de função, a delegação de chamadas — notadamente a thread1 invocando atualizar_banco() , a qual, por seu turno, aciona registrar_log() — ofusca a cadeia de dependências e oculta a real ordem de retenção dos mutexes. Essa abstração mascara o risco estrutural de intertravamento durante a análise estática do código.

Rb.Q3.: Sim, ocorrerá irremediavelmente um impasse restrito, especificamente classificado como auto-bloqueio (self-deadlock). A função thread1 adquire e retém o log_mutex para, em seguida, invocar a função atualizar_banco(). Esta última rotina adquire o banco_mutex e procede com a invocação de registrar_log() , a qual exige a aquisição primária do log_mutex. Como as implementações padrão dos mutexes na biblioteca POSIX não possuem

propriedade recursiva por omissão, a thread suspenderá sua própria execução de modo perene, aguardando a liberação de um bloqueio que ela própria já detém.

Rc.Q3.: Abstraindo-se o auto-bloqueio imanente e hipotetizando que o mutex suportasse recursividade, um cenário clássico de impasse (espera circular) manifestar-se-ia plenamente mediante a introdução de uma thread concorrente que solicitasse os recursos de forma invertida. Uma thread2 proposta adquiriria inicialmente o banco_mutex e, incontinenti, tentaria reter o log_mutex de maneira direta. Se a thread2 efetuasse seu bloqueio no interregno em que a thread1 retém o log_mutex mas ainda não alcançou a instrução sobre o banco_mutex, ambas incorreriam em intertravamento absoluto mútuo.

Rd.Q3.:

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-auxiliar-function-wich-hides-risk.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

// Initialization of the global synchronization primitives
pthread_mutex_t log_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t banco_mutex = PTHREAD_MUTEX_INITIALIZER;

void registrar_log()
{
    // Safely acquires the log_mutex.
    // In the corrected architecture, the calling thread does not hold it
    beforehand.
    pthread_mutex_lock(&log_mutex);
    usleep(1000);
    printf("Log registrado com sucesso.\n");
    pthread_mutex_unlock(&log_mutex);
}

void atualizar_banco()
{

```



```

    // Acquires the database mutex to protect the database critical
section
    pthread_mutex_lock(&banco_mutex);
    usleep(1000);
    printf("Banco de dados atualizado.\n");

    // It is mathematically safe to invoke registrar_log() here because
    // the caller function (thread1) no longer holds log_mutex, thereby
    // neutralizing the self-deadlock vulnerability.
    registrar_log();

    pthread_mutex_unlock(&banco_mutex);
}

void *thread1(void *arg)
{
    pthread_mutex_lock(&log_mutex);
    usleep(1000);
    printf("Thread 1: Executando procedimento primario de log.\n");

    // FIX: The core vulnerability was maintaining the possession of
log_mutex
    // while executing atualizar_banco(). By releasing the lock strictly
BEFORE
    // the nested function call, we intrinsically prevent the self-
deadlock.
    pthread_mutex_unlock(&log_mutex);

    atualizar_banco();

    return NULL;
}

// Proposed second thread to demonstrate safe concurrent execution
void *thread2(void *arg)
{
    // Thread 2 attempts to perform the database update directly.
    // Due to the structural fix in thread1, there is no circular wait
condition.
    printf("Thread 2: Iniciando operacao independente no banco.\n");
    atualizar_banco();

    return NULL;
}

int main()
{
    pthread_t t1, t2;

```

```

// Instantiating and dispatching the threads concurrently
pthread_create(&t1, NULL, thread1, NULL);
pthread_create(&t2, NULL, thread2, NULL);

// Synchronizing the main process execution with the completion of
the threads
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Todas as execucoes foram concluidas de forma integra e sem
intertravamentos.\n");

return 0;
}

```

Questão 4 – *trylock* e risco de *livelock*

Ra.Q4.: Não ocorre impasse (deadlock) clássico. A consecução de um intertravamento absoluto demanda a verificação simultânea e incontornável de quatro propriedades fundamentais, dentre as quais a "posse e espera" incondicional. Na arquitetura apresentada, caso uma thread não logre êxito na obtenção não-bloqueante do segundo recurso mediante a primitiva `pthread_mutex_trylock`, ela libera proativamente o primeiro recurso detido por meio da instrução `pthread_mutex_unlock`. Essa concessão voluntária inviabiliza o bloqueio mútuo e perpétuo característico do deadlock.

Rb.Q4.: Sim, o arranjo lógico exibe flagrante suscetibilidade ao fenômeno denominado livelock (bloqueio ativo). Essa anomalia de concorrência manifesta-se caso ambas as threads (t1 e t2) alcancem uma cadência de execução perfeitamente simétrica. Ambas adquirem seu respectivo mutex primário simultaneamente, falham na tentativa cruzada de aquisição do secundário, abdicam de seus recursos, suspendem suas atividades pelo mesmo interregno de tempo via `usleep(100)` e reiniciam o ciclo iterativo de forma concomitante. O resultado é a perpetuação de operações de contenção sem que haja progresso computacional substantivo ou efetivação das seções críticas.

Rc.Q4.: As condições mitigadas pela implementação do laço de reiteração e do descarte de recursos são a "posse e espera" e a "espera circular". A unidade de processamento não retém o recurso primário de forma ociosa enquanto demanda o secundário, e, conseqüentemente, não encadeia um ciclo inquebrável de aguardo mútuo. Permanecem vigentes, contudo, as propriedades de "exclusão mútua" e "não-preempção", porquanto os mutexes declarados continuam a garantir acesso serializado intransferível e o sistema operacional subjacente abstém-se de revogar coercitivamente os bloqueios legitimamente outorgados.

Rd.Q4.: A retificação plena desta vulnerabilidade repousa na substituição da heurística de contenção baseada em tentativas transacionais ativas por um modelo de ordenação determinístico. A aplicação de uma hierarquia global e estrita para a aquisição paralela de recursos erradica matematicamente tanto o risco de deadlock quanto a entropia do livelock.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-trylock-and-livelock-risk.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

// Initialization of the concurrent synchronization objects
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

void *t1(void *arg)
{
    // Establishing a strict, global lock acquisition hierarchy: m1 is
    // locked prior to m2.
    pthread_mutex_lock(&m1);
    pthread_mutex_lock(&m2);

    // Critical section execution sequence
    printf("t1 executando \n");

    // Relinquishing system resources in the mathematically inverse order
    // of acquisition
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
}
```

```

        return NULL;
    }

void *t2(void *arg)
{
    // FIX: The livelock vulnerability was intrinsically bound to the
    non-blocking polling
    // mechanism coupled with inverse resource requests. By forcing t2 to
    strictly comply
    // with the universal locking order (m1 then m2), we algorithmically
    neutralize
    // any potential for both livelock regressions and classical
    deadlocks.
    pthread_mutex_lock(&m1);
    pthread_mutex_lock(&m2);

    // Critical section execution sequence
    printf("t2 executando \n");

    // Relinquishing system resources
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    // Instantiating the execution threads
    pthread_create(&thread1, NULL, t1, NULL);
    pthread_create(&thread2, NULL, t2, NULL);

    // Main process synchronization barrier
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Concorrença executada e estabilizada sem incidentes de
    impasses ativados ou passivos.\n");

    return 0;
}

```

Questão 5 – Recursos adquiridos em função recursiva

Ra.Q5.: Ocorrem ambos os fenómenos. O auto-bloqueio (self-deadlock) consubstancia-se na thread1, porquanto a rotina processar é recursiva e postula a retenção do recurso A sucessivas vezes sem o ter libertado previamente. Como os mutexes da norma POSIX não são recursivos por omissão, a thread suspende a sua própria execução de forma perpétua. Simultaneamente, existe a suscetibilidade a um impasse entre threads (inter-thread deadlock). Caso a thread1 adquira o recurso A na primeira iteração e, em paralelo, a thread2 adquira o recurso B, estabelecer-se-á uma espera circular irresolúvel quando a thread1 solicitar B e a thread2 demandar A.

Rb.Q5.: A recursividade ofusca a linearidade da aquisição de bloqueios durante a análise estática do código. O aninhamento das invocações da sub-rotina oculta o facto de que a primitiva de exclusão mútua (`pthread_mutex_lock(&A)`) será executada múltiplas vezes pelo mesmo fluxo de controlo antes que a instrução de libertação (`pthread_mutex_unlock(&A)`) seja sequer alcançada no desempilhamento das chamadas, camuflando a vulnerabilidade arquitetural do auto-bloqueio.

Rc.Q5.: As quatro condições necessárias e suficientes de Coffman encontram-se rigorosamente presentes para o impasse inter-threads: a exclusão mútua (inerente à natureza estrutural dos mutexes), a posse e espera (a thread1 detém A e aguarda B; a thread2 detém B e aguarda A), a não-preempção (o sistema operativo não revoga coercivamente os bloqueios atribuídos) e a espera circular (a interdependência cíclica e cruzada das requisições). No âmbito restrito do auto-bloqueio, constata-se a exclusão mútua, a posse e espera e a não-preempção sobre o mesmo recurso por uma única unidade de processamento.

Rd.Q5.: Para erradicar ambas as patologias concorrentes, impõe-se uma dupla intervenção: primeiro, o recurso A deve ser instanciado explicitamente com o atributo de mutex recursivo, permitindo reentrância segura pela mesma thread; segundo, a thread2 deve subordinar-se a uma ordem global estrita de aquisição

(adquirindo A e, subsequentemente, B), rompendo matematicamente a condição de espera circular.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-resources-getted-in-recursive-function.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Global declaration of mutexes.
// Mutex A will be dynamically initialized as recursive in main() to
prevent self-deadlocks.
pthread_mutex_t A;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void processar(int nivel)
{
    if (nivel == 0)
        return;

    // The thread recursively acquires mutex A.
    // Because A is now explicitly configured as a recursive mutex,
    // the calling thread can lock it multiple times without triggering a
self-deadlock.
    pthread_mutex_lock(&A);
    usleep(1000);

    if (nivel % 2 == 0)
    {
        // Acquiring B after A naturally follows the designated global
locking hierarchy.
        pthread_mutex_lock(&B);
        pthread_mutex_unlock(&B);
    }

    // Recursive call: safe reentrancy is now guaranteed.
    processar(nivel - 1);

    // Each successful lock operation on a recursive mutex must be
matched by an unlock operation.
    pthread_mutex_unlock(&A);
}

void *thread1(void *arg)
```

```

{
    processar(2);
    return NULL;
}

void *thread2(void *arg)
{
    // FIX: Mitigating the inter-thread deadlock by adhering to a strict
    global locking sequence.
    // Thread 2 must acquire resource A prior to B, unconditionally
    mirroring the acquisition order
    // implicitly established within the 'processar' function of thread
    1.
    pthread_mutex_lock(&A);
    usleep(1000);
    pthread_mutex_lock(&B);

    // Critical section execution sequence
    printf("Thread 2 executada com segurança.\n");

    // Relinquishing system resources in the reverse mathematical order
    of their acquisition
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);

    return NULL;
}

int main()
{
    // Configuring mutex A with recursive attributes dynamically
    pthread_mutexattr_t attr;
    if (pthread_mutexattr_init(&attr) != 0)
    {
        perror("Falha ao inicializar atributos do mutex");
        exit(EXIT_FAILURE);
    }

    if (pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE) != 0)
    {
        perror("Falha ao definir o tipo recursivo");
        exit(EXIT_FAILURE);
    }

    if (pthread_mutex_init(&A, &attr) != 0)
    {
        perror("Falha ao inicializar o mutex A");
        exit(EXIT_FAILURE);
    }
}

```

```

pthread_t t1, t2;

// Dispatching the concurrent execution threads
pthread_create(&t1, NULL, thread1, NULL);
pthread_create(&t2, NULL, thread2, NULL);

// Synchronizing the main process barrier
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Execucao concluida com sucesso, estritamente livre de auto-
bloqueios e impasses inter-threads.\n");

// Reclaiming allocated system resources
pthread_mutexattr_destroy(&attr);
pthread_mutex_destroy(&A);
pthread_mutex_destroy(&B);

return 0;
}

```

Questão 6 – Produtor, manutenção e função auxiliar

Ra.Q6.: Sim, a ocorrência do impasse encontra-se intrinsecamente subordinada às decisões de escalonamento efetuadas pelo sistema operativo. O intertravamento mútuo manifesta-se de forma exclusiva caso a preempção determine que a thread produtor adquira o fila_mutex e, durante o interregno induzido pela função usleep, o escalonador transira o controlo do processador para a thread manutencao, permitindo-lhe reter o estat_mutex. Na eventualidade de uma das threads executar a sua secção crítica na íntegra sem sobreposição temporal com a outra, a execução do programa prosseguirá sem qualquer anomalia.

Rb.Q6.: Sim, a referida sub-rotina é instrumental para a consumação da condição de posse e espera. Ao ser invocada no escopo da thread produtor — momento em que esta já detém o bloqueio sobre o fila_mutex —, a função exige a aquisição de um recurso adicional, o estat_mutex. Consequentemente, a linha de execução mantém ativamente a posse de um recurso primário enquanto

aguarda a concessão de um secundário, satisfazendo plenamente a condição arquitetural de hold and wait.

Rc.Q6.: Para mitigar esta vulnerabilidade concorrente, destacam-se duas estratégias estruturais distintas:

1. **Imposição de uma hierarquia global de aquisição:** Consiste em padronizar, de forma dogmática, a sequência de bloqueios em todo o sistema. Se for estipulado que o fila_mutex deve ser sempre adquirido antes do estat_mutex, a thread manutencao deverá ser reestruturada para respeitar esta ordem, erradicando a possibilidade de espera circular.
2. **Libertação antecipada de recursos:** Consiste em reestruturar a thread produtor para que esta liberte o fila_mutex estritamente antes de invocar a rotina independente atualizar_estatisticas(). Esta abordagem suprime a condição de posse e espera, garantindo o progresso do sistema de forma segura.

Rd.Q6.: A solução apresentada abaixo implementa a segunda forma de prevenção (libertação antecipada), por ser arquiteturalmente mais limpa quando se lida com funções auxiliares encapsuladas, reduzindo adicionalmente o tempo de contenção do recurso principal.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-production-maintenance-and-auxiliar-function.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Initialization of global synchronization primitives
pthread_mutex_t fila_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t estat_mutex = PTHREAD_MUTEX_INITIALIZER;

void atualizar_estatisticas()
{
```

```

    // The function safely acquires its required lock
    pthread_mutex_lock(&estat_mutex);
    usleep(1000);
    printf("Estatisticas atualizadas com sucesso.\n");
    pthread_mutex_unlock(&estat_mutex);
}

void *produtor(void *arg)
{
    // Acquires the primary resource
    pthread_mutex_lock(&fila_mutex);
    usleep(1000);
    printf("Produtor: Operacoes na fila concluidas.\n");

    // FIX: Early release of the resource.
    // By unlocking the fila_mutex BEFORE calling the auxiliary function,
    // we structurally eliminate the "hold and wait" condition.
    // The thread no longer holds a lock while requesting another.
    pthread_mutex_unlock(&fila_mutex);

    // Calls the auxiliary function in a safe, unencumbered state
    atualizar_estatisticas();

    return NULL;
}

void *manutencao(void *arg)
{
    // The maintenance thread can now safely acquire its locks.
    // Since the producer no longer holds the fila_mutex while
    // holding/requesting
    // the estat_mutex, the circular wait condition is mathematically
    // impossible here.
    pthread_mutex_lock(&estat_mutex);
    usleep(1000);
    pthread_mutex_lock(&fila_mutex);

    printf("Manutencao: Rotina de verificacao executada.\n");

    // Releasing locks in the reverse order of acquisition
    pthread_mutex_unlock(&fila_mutex);
    pthread_mutex_unlock(&estat_mutex);

    return NULL;
}

int main()
{
    pthread_t thread_produtor, thread_manutencao;

```

```

// Dispatching the producer and maintenance threads concurrently
if (pthread_create(&thread_produto, NULL, produtor, NULL) != 0)
{
    perror("Erro ao criar a thread produtora");
    exit(EXIT_FAILURE);
}

if (pthread_create(&thread_manutencao, NULL, manutencao, NULL) != 0)
{
    perror("Erro ao criar a thread de manutencao");
    exit(EXIT_FAILURE);
}

// Synchronizing the main execution thread with the concurrent
workers
pthread_join(thread_produto, NULL);
pthread_join(thread_manutencao, NULL);

printf("Execucao integralizada sem registo de intertravamentos.\n");

// Resource cleanup
pthread_mutex_destroy(&fila_mutex);
pthread_mutex_destroy(&estat_mutex);

return 0;
}

```

Questão 7 – Impressora e spooler com modelagem incorreta

Ra.Q7.: Sim, constata-se uma iminente e severa possibilidade de impasse (deadlock). A anomalia arquitetural decorre da ordem invertida na requisição dos recursos de exclusão mútua. A thread usuario1 adquire o acesso exclusivo à impressora e, subsequentemente, suspende sua execução no aguardo da concessão do spooler. Em contrapartida, a thread usuario2 apodera-se do spooler e demanda o acesso à impressora. Caso a preempção do sistema operativo intercale a execução de ambas as threads exatamente após as suas primeiras requisições, o sistema estagnarà num intertravamento mútuo e perpétuo, configurando uma espera circular irresolúvel.

Rb.Q7.: A modelagem apresentada subverte a premissa fundacional do mecanismo de spooling (Simultaneous Peripheral Operations On-line). O

propósito basilar de um spooler é atuar como um intermediário assíncrono (um buffer ou fila) para desvincular os processos de utilizador da interação direta com dispositivos periféricos intrinsecamente lentos, como impressoras. Na arquitetura correta, os utilizadores submetem os seus trabalhos exclusivamente ao spooler e prosseguem com as suas rotinas. Cabe a um processo independente, operando em segundo plano (frequentemente designado como daemon de impressão), a responsabilidade exclusiva de extrair os trabalhos do spooler e interagir diretamente com o hardware da impressora.

Rc.Q7.: As quatro condições necessárias e suficientes postuladas por Coffman manifestam-se inequivocamente na estrutura do código:

1. **Exclusão mútua:** Os recursos (impressora e spooler) são protegidos por mutexes, garantindo acesso serializado e intransferível.
2. **Posse e espera (Hold and Wait):** Cada thread retém legitimamente um recurso primário enquanto postula ativamente a concessão de um recurso secundário.
3. **Não-preempção:** O ambiente POSIX não preempata coercitivamente os locks; um recurso só é libertado voluntariamente pela thread detentora.
4. **Espera circular:** Estabelece-se um ciclo topológico fechado de dependências, no qual usuario1 aguarda por usuario2, que, por seu turno, aguarda por usuario1.

Rd.Q7.: Para sanar a vulnerabilidade sistêmica e respeitar o paradigma de spooling, os utilizadores devem interagir única e exclusivamente com o mutex do spooler. Um daemon dedicado assumirá o papel de consumidor, consultando o spooler e adquirindo o mutex da impressora de forma controlada.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-printer-and-spooner-with-incorrect-model.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Initialization of global synchronization primitives
pthread_mutex_t impressora = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t spooler_mutex = PTHREAD_MUTEX_INITIALIZER;

// Simulated shared resource representing the spooler queue
int trabalhos_no_spoiler = 0;

void *usuario1(void *arg)
```

```

{
    // FIX: Users no longer interact with the printer hardware directly.
    // They only acquire the spooler lock to submit their jobs safely.
    pthread_mutex_lock(&spooler_mutex);
    usleep(500); // Simulating time taken to enqueue a document
    trabalhos_no_spooler++;
    printf("Usuario 1: Documento submetido com sucesso ao spooler.\n");
    pthread_mutex_unlock(&spooler_mutex);

    return NULL;
}

void *usuario2(void *arg)
{
    // Identical safe architecture for the second user thread.
    pthread_mutex_lock(&spooler_mutex);
    usleep(500);
    trabalhos_no_spooler++;
    printf("Usuario 2: Documento submetido com sucesso ao spooler.\n");
    pthread_mutex_unlock(&spooler_mutex);

    return NULL;
}

// Independent background process representing the Spooler Daemon
void *daemon_spooler(void *arg)
{
    int trabalhos_pendentes = 0;

    // The daemon safely checks the spooler queue
    pthread_mutex_lock(&spooler_mutex);
    if (trabalhos_no_spooler > 0)
    {
        trabalhos_pendentes = trabalhos_no_spooler;
        trabalhos_no_spooler = 0; // Clears the queue after fetching jobs
    }
    pthread_mutex_unlock(&spooler_mutex);

    // If there are jobs, the daemon is the ONLY entity that directly
    locks the printer
    if (trabalhos_pendentes > 0)
    {
        pthread_mutex_lock(&impressora);
        printf("Daemon do Spooler: Imprimindo %d trabalho(s)
pendente(s)...\n", trabalhos_pendentes);
        usleep(1500); // Simulating hardware printing time
        printf("Daemon do Spooler: Impressao concluida de forma
segura.\n");
        pthread_mutex_unlock(&impressora);
    }
}

```

```

    }
    else
    {
        printf("Daemon do Spooler: Nenhum trabalho pendente na fila.\n");
    }

    return NULL;
}

int main()
{
    pthread_t t_usr1, t_usr2, t_daemon;

    // Dispatching user threads concurrently to simulate simultaneous job
    submission
    pthread_create(&t_usr1, NULL, usuario1, NULL);
    pthread_create(&t_usr2, NULL, usuario2, NULL);

    // Synchronizing user completion before invoking the daemon for
    demonstration purposes
    pthread_join(t_usr1, NULL);
    pthread_join(t_usr2, NULL);

    // Launching the daemon to process the accumulated spooler queue
    pthread_create(&t_daemon, NULL, daemon_spooler, NULL);
    pthread_join(t_daemon, NULL);

    printf("Operacoes de spooling e impressao finalizadas sem
    intertravamentos.\n");

    // Resource cleanup
    pthread_mutex_destroy(&impressora);
    pthread_mutex_destroy(&spooler_mutex);

    return 0;
}

```

Questão 8 – Deadlock oculto por abstração em módulo

Ra.Q8.: A modularização e o encapsulamento das primitivas de sincronização em sub-rotinas distintas ofuscam a cadeia real de dependências de recursos. Durante a análise estática ou revisão da função `atualizar_cache`, o desenvolvedor não visualiza explicitamente a aquisição do `disco_mutex` — omitida topologicamente pela invocação da função `gravar_disco()`. Esta abstração arquitetural dissimula a perigosa inversão na ordem de retenção dos

bloqueios face à lógica implementada em `flush_disco_para_cache`, tornando a falha invisível em análises superficiais do escopo local.

Rb.Q8.: O intertravamento materializa-se quando uma `thread_A` invoca `atualizar_cache()` de forma estritamente concomitante a uma `thread_B` que executa `flush_disco_para_cache()`. A `thread_A` adquire primariamente o `cache_mutex` e, por conseguinte, sofre preempção por parte do sistema operativo. No seu interregno de tempo de processamento, a `thread_B` adquire o `disco_mutex` e suspende-se ao tentar obter o `cache_mutex`, já reservado. Quando a `thread_A` retoma a execução e atinge a rotina `gravar_disco()`, postula o acesso ao `disco_mutex` (retido ativamente pela `thread_B`), consolidando um cenário de impasse absoluto, simétrico e irresolúvel.

Rc.Q8.: As quatro condições necessárias e suficientes postuladas por Coffman manifestam-se de maneira taxativa:

1. **Exclusão mútua:** Assegurada inexoravelmente pela primitiva `pthread_mutex_t`, que defere o acesso singular e intransferível quer ao cache, quer ao disco.
2. **Posse e espera:** Cada thread preserva o domínio sobre o seu recurso basal (cache ou disco, respetivamente) enquanto emite uma requisição bloqueante para a concessão do recurso secundário complementar.
3. **Não-preempção:** O ambiente de concorrência POSIX inviabiliza a expropriação forçada de um bloqueio; o recurso apenas retorna à disponibilidade perante libertação voluntária.
4. **Espera circular:** Consuma-se um ciclo perfeitamente fechado de interdependências, no qual a primeira unidade de processamento aguarda a segunda, que, paradoxalmente, aguarda a primeira.

Rd.Q8.: A profilaxia arquitetural para esta vulnerabilidade requer a estipulação dogmática de uma ordem global de aquisição de recursos. Arbitra-se convencionalmente que o bloqueio do cache deverá preceder, invariavelmente, o bloqueio do disco. Consequentemente, a função `flush_disco_para_cache` necessita ser reestruturada para alinhar-se à sequência implícita de `atualizar_cache` (cache antecedendo disco), mitigando em definitivo a possibilidade matemática de intersecções cíclicas.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-hide-deadlock-bypass-model.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Global synchronization primitives defining the shared resources
pthread_mutex_t cache_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t disco_mutex = PTHREAD_MUTEX_INITIALIZER;

void gravar_disco()
{
    // The disk mutex is acquired safely based on the caller's
    // established hierarchy
    pthread_mutex_lock(&disco_mutex);
    usleep(1000);
    printf("Operacao de I/O concluida: Dado gravado no disco.\n");
    pthread_mutex_unlock(&disco_mutex);
}

void atualizar_cache()
{
    // Acquires the cache mutex first.
    // The effective global lock order here implicitly becomes: Cache ->
    // Disk.
    pthread_mutex_lock(&cache_mutex);
    usleep(1000);
    printf("Memoria transitoria: Cache atualizado.\n");

    // Calls the external function which will subsequently lock the disk
    // mutex
    gravar_disco();

    pthread_mutex_unlock(&cache_mutex);
}
```



```

void flush_disco_para_cache()
{
    // FIX: Architectural compliance with the global locking hierarchy.
    // To eradicate the hidden deadlock, this function MUST acquire the
cache mutex
    // BEFORE the disk mutex, mirroring the exact order established by
atualizar_cache().
    pthread_mutex_lock(&cache_mutex);
    pthread_mutex_lock(&disco_mutex);

    usleep(1000);
    printf("Procedimento de flush concluido com integridade
estrutural.\n");

    // Relinquishing system resources symmetrically in the reverse order
of acquisition
    pthread_mutex_unlock(&disco_mutex);
    pthread_mutex_unlock(&cache_mutex);
}

// Auxiliary thread functions to demonstrate concurrent execution
void *thread_cache(void *arg)
{
    atualizar_cache();
    return NULL;
}

void *thread_flush(void *arg)
{
    flush_disco_para_cache();
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    // Instantiating the execution threads concurrently
    if (pthread_create(&t1, NULL, thread_cache, NULL) != 0)
    {
        perror("Falha ao instanciar thread_cache");
        exit(EXIT_FAILURE);
    }
    if (pthread_create(&t2, NULL, thread_flush, NULL) != 0)
    {
        perror("Falha ao instanciar thread_flush");
        exit(EXIT_FAILURE);
    }
}

```

```

// Synchronizing the main process execution boundary
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("Todas as transacoes de modulos foram concluídas estritamente
sem intertravamentos.\n");

// Graceful destruction of the synchronization objects
pthread_mutex_destroy(&cache_mutex);
pthread_mutex_destroy(&disco_mutex);

return 0;
}

```

Questão 9 – Timeout parcial e solução inconsistente

Ra.Q9.: Não ocorre um impasse (deadlock) clássico, uma vez que a circularidade é interrompida. Todavia, emerge um grave problema de concorrência consubstanciado no abandono definitivo de tarefa (uma forma extrema de inanição ou starvation). Quando a thread t1 falha na obtenção do recurso secundário (y) através da primitiva não-bloqueante, ela liberta o recurso primário e termina imediatamente a sua execução (return NULL), abdicando perpetuamente de concretizar a sua secção crítica e comprometendo a lógica aplicacional.

Rb.Q9.: Não, a resolução é manifestamente paliativa, assimétrica e estruturalmente inconsistente. Embora evite o colapso absoluto do sistema operativo através da abdicação voluntária por parte de t1, não garante o progresso efetivo de todos os processos envolvidos. A integridade sistémica fica lesada, visto que a arquitetura delega a responsabilidade de prevenção do impasse ao sacrifício unilateral de uma das threads, resultando num sistema onde operações essenciais podem ser ignoradas e descartadas sem qualquer mecanismo de recuperação ou repetição.

Rc.Q9.: Para a thread t1, mitiga-se inequivocamente a condição de posse e espera (hold and wait), dado que liberta o recurso x em caso de insucesso na

requisição de y. Consequentemente, a espera circular é inviabilizada de uma perspectiva global. Em contrapartida, a thread t2 preserva intactas na sua conceção local as quatro condições clássicas, mantendo ativamente a posse e espera ao reter y enquanto invoca uma primitiva estritamente bloqueante e incondicional sobre x.

Rd.Q9.: A profilaxia adequada e consistente para este cenário não passa pela introdução de abstenções assimétricas (trylock), mas sim pela subordinação estrita e simétrica de ambas as threads a uma hierarquia global de aquisição de recursos (bloquear x antecedendo sempre y).

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/parcial-timeout-and-inconsistent-solution.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Initialization of the global synchronization primitives
pthread_mutex_t x = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t y = PTHREAD_MUTEX_INITIALIZER;

void *t1(void *arg)
{
    // Thread 1 strictly follows the global lock acquisition hierarchy: x
    // then y.
    pthread_mutex_lock(&x);
    sleep(1);

    // The trylock and task abandonment logic has been structurally
    // removed.
    // By relying on the global order, a standard blocking lock is now
    // mathematically safe.
    pthread_mutex_lock(&y);

    printf("Thread t1 concluiu a sua seccao critica com integridade.\n");

    // Relinquishing system resources symmetrically in the reverse order
    pthread_mutex_unlock(&y);
    pthread_mutex_unlock(&x);
}
```

```

        return NULL;
    }

void *t2(void *arg)
{
    // FIX: Architectural compliance with the global locking hierarchy.
    // To eradicate any possibility of deadlock without resorting to
    asymmetrical thread starvation,
    // t2 MUST acquire mutex 'x' BEFORE mutex 'y', mirroring the order
    established in t1.
    pthread_mutex_lock(&x);
    sleep(1);
    pthread_mutex_lock(&y);

    printf("Thread t2 concluiu a sua seccao critica com integridade.\n");

    // Relinquishing system resources
    pthread_mutex_unlock(&y);
    pthread_mutex_unlock(&x);

    return NULL;
}

int main()
{
    pthread_t thread_1, thread_2;

    // Instantiating the concurrent execution threads
    if (pthread_create(&thread_1, NULL, t1, NULL) != 0)
    {
        perror("Falha na alocao da thread t1");
        exit(EXIT_FAILURE);
    }
    if (pthread_create(&thread_2, NULL, t2, NULL) != 0)
    {
        perror("Falha na alocao da thread t2");
        exit(EXIT_FAILURE);
    }

    // Main process synchronization barrier ensuring full completion
    pthread_join(thread_1, NULL);
    pthread_join(thread_2, NULL);

    printf("Sistema convergido: Todas as transacoes foram executadas sem
    abandono de tarefas.\n");

    // Graceful destruction of the synchronization objects
    pthread_mutex_destroy(&x);
    pthread_mutex_destroy(&y);

```

```
return 0;  
}
```

Questão 10 – Múltiplos recursos com ciclo de dependência

Ra.Q10.: Sim, constata-se a formação irrefutável de múltiplos ciclos de espera. O sistema exhibe impasses bidirecionais isolados: a thread t1 retém o recurso a e aguarda a concessão de b, enquanto a t3 retém b e demanda a, estabelecendo um ciclo fechado. Paralelamente, na eventualidade de a t1 progredir, reter b e postular c, entrará em colisão direta com a t2, que retém c e exige b, materializando um segundo ciclo estrutural de intertravamento independente do primeiro.

Rb.Q10.: A circularidade completa pressupõe uma topologia em que todas as entidades do sistema formam um único anel ininterrupto e interdependente (por exemplo, t1 aguarda t2, que aguarda t3, que por sua vez aguarda t1). Neste cenário específico, prevalecem as dependências parciais: os impasses formam-se em subconjuntos restritos de recursos e processos. O bloqueio entre t1 e t3 (disputando a e b) ocorre de forma perfeitamente alheia à existência do recurso c ou da thread t2. Trata-se, pois, de micro-circularidades localizadas que colapsam a execução de forma fragmentada, consubstanciando uma dependência parcial, em detrimento de uma falha global unificada.

Rc.Q10.: As quatro condições necessárias e suficientes postuladas por Coffman manifestam-se de modo taxativo:

1. **Exclusão mútua:** Assegurada pela natureza inerente das primitivas `pthread_mutex_t`, que impõem acesso serializado e singular a cada recurso (a, b e c).
2. **Posse e espera (Hold and Wait):** Cada thread preserva ativamente a tutela sobre um recurso previamente adquirido (ex.: t1 detém a) durante

a submissão de um pedido estritamente bloqueante por um recurso adicional (ex.: b).

3. **Não-preempção:** O ambiente de concorrência POSIX e o sistema operativo subjacente abstêm-se de expropriações coercivas dos bloqueios vigentes; a libertação de um recurso é prerrogativa exclusiva e voluntária da thread detentora.
4. **Espera circular:** Consubstanciada nos referidos ciclos parciais de dependência mútua evidenciados, nos quais a contenção paralela inviabiliza o progresso algorítmico global.

Rd.Q10.: A profilaxia absoluta contra esta anomalia multivariada requer a instituição de uma hierarquia universal e dogmática de aquisição (neste caso, a ordem alfabética: a precedendo b, e b precedendo c). Ao forçar todas as *threads* a respeitarem esta sequência rigorosa, erradica-se matematicamente a possibilidade de interseções cíclicas.

Também disponível em: <https://github.com/L1K3D/detection-and-prevention-of-impasses/blob/main/corrected-c-scripts/ccs-multiple-resources-with-dependence-cicle.c>

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

// Initialization of the global synchronization primitives
pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t c = PTHREAD_MUTEX_INITIALIZER;

void *t1(void *arg)
{
    // Thread 1 inherently respects the global hierarchical order: A -> B
    // -> C
    pthread_mutex_lock(&a);
    usleep(1000);
    pthread_mutex_lock(&b);
    usleep(1000);
    pthread_mutex_lock(&c);
```

```

    // Critical section execution
    printf("Thread t1 executada com sucesso.\n");

    // Relinquishing resources in the exact mathematical reverse order of
    their acquisition
    pthread_mutex_unlock(&c);
    pthread_mutex_unlock(&b);
    pthread_mutex_unlock(&a);

    return NULL;
}

void *t2(void *arg)
{
    // FIX: Architectural compliance with the established global order.
    // To prevent the partial dependency cycle with t1, thread 2 MUST
    acquire
    // mutex 'b' strictly BEFORE mutex 'c'.
    pthread_mutex_lock(&b);
    usleep(1000);
    pthread_mutex_lock(&c);

    // Critical section execution
    printf("Thread t2 executada com sucesso.\n");

    // Releasing locks sequentially backwards
    pthread_mutex_unlock(&c);
    pthread_mutex_unlock(&b);

    return NULL;
}

void *t3(void *arg)
{
    // FIX: Eradicating the direct circular wait condition with t1.
    // Thread 3 MUST acquire mutex 'a' before mutex 'b', adhering to the
    global state hierarchy.
    pthread_mutex_lock(&a);
    usleep(1000);
    pthread_mutex_lock(&b);

    // Critical section execution
    printf("Thread t3 executada com sucesso.\n");

    // Releasing locks symmetrically
    pthread_mutex_unlock(&b);
    pthread_mutex_unlock(&a);
}

```

```

        return NULL;
    }

int main()
{
    pthread_t thread_1, thread_2, thread_3;

    // Dispatching the concurrent execution threads
    if (pthread_create(&thread_1, NULL, t1, NULL) != 0)
    {
        perror("Falha na alocação da thread t1");
        exit(EXIT_FAILURE);
    }
    if (pthread_create(&thread_2, NULL, t2, NULL) != 0)
    {
        perror("Falha na alocação da thread t2");
        exit(EXIT_FAILURE);
    }
    if (pthread_create(&thread_3, NULL, t3, NULL) != 0)
    {
        perror("Falha na alocação da thread t3");
        exit(EXIT_FAILURE);
    }

    // Synchronizing the main process barrier, ensuring full systemic
completion
    pthread_join(thread_1, NULL);
    pthread_join(thread_2, NULL);
    pthread_join(thread_3, NULL);

    printf("Sistema convergido: Todas as transações foram executadas sem
presença de ciclos mortos.\n");

    // Graceful destruction of the synchronization objects to prevent
memory leaks
    pthread_mutex_destroy(&a);
    pthread_mutex_destroy(&b);
    pthread_mutex_destroy(&c);

    return 0;
}

```


Conclusão

Em síntese, a análise e retificação da presente bateria de exercícios demonstram inequivocamente que a programação concorrente, embora indispensável para a otimização de desempenho em Sistemas Operativos, encerra vulnerabilidades arquiteturais severas quando não é rigorosamente disciplinada. A manipulação de recursos partilhados através da biblioteca POSIX Threads (pthread) exige um controlo estrito e proativo sobre as quatro condições de Coffman, sob pena de colapso sistémico.

Ao longo da resolução dos cenários propostos, ficou patente que a esmagadora maioria das patologias de concorrência — sejam impasses clássicos (*deadlocks*), auto-bloqueios (*self-deadlocks*), bloqueios ativos (*livelocks*) ou anomalias de abandono de tarefas (*starvation*) — deriva da ausência de uma taxonomia clara e unificada na requisição de bloqueios de exclusão mútua (*mutexes*). A estratégia transversal, profilática e mais robusta aplicada para sanar estas vulnerabilidades consistiu na subordinação de todas as *threads* a uma **ordem global e determinística de aquisição de recursos**. Ao forçar o sistema a respeitar uma hierarquia universal, rompeu-se matematicamente a possibilidade de ocorrência de ciclos de espera.

Adicionalmente, o estudo evidenciou a importância de outras boas práticas de engenharia de *software* de baixo nível, tais como a libertação antecipada de *mutexes* antes da invocação de sub-rotinas opacas (mitigando a posse e espera) e o respeito pelos paradigmas clássicos de arquitetura de sistemas (como a delegação de interações de *hardware* a *daemons* dedicados, em vez de acessos diretos pelo utilizador). Por fim, atestou-se que soluções baseadas em primitivas não-bloqueantes (trylock) ou dependentes do comportamento do escalonador são, na sua essência, paliativas e assimétricas, não substituindo a integridade de um bom planeamento estrutural.

Em suma, conclui-se que um sistema concorrente robusto deve ser arquiteturalmente imune a intertravamentos por conceção (*by design*), garantindo a integridade, a simetria e o progresso contínuo de todas as operações, independentemente das flutuações temporais ou das decisões de preempção impostas pelo sistema operativo.